同濟大學
TONGJI UNIVERSITY

Python 人工智能程序设计期中试题 2

——基于模拟退火的旅行商问题求解

姓　　名：　杨玺威

学　　号：　2453360

专　　业：　人工智能

二〇二六年四月

1 问题描述

题目 2：假设一位旅行商需要访问下列这些城市，并最终返回起始城市，且每个城市仅经过一次。给定以下城市坐标，编写 Python 代码找到最短的访问路径。

城市	X	Y
北京	9932	4439
天津	10109	4351
上海	11552	3472

总共有 34 个省会、民族自治区首府、直辖市或特别行政区。

本问题为经典旅行商问题（TSP），属于 NP 难组合优化问题，目标是遍历所有城市并返回起点，使总路径最短。当城市数量较少时，可以通过暴力枚举所有排列求得精确解；但随着城市数增长，计算复杂度呈阶乘级爆炸，根据图论的知识，n 个城市可能的路径总数为$\frac{(n-1)!}{2}$条，因此，当 n=34 时，路径总数已高达 10^{38} 量级，已远超现有计算机的枚举计算能力。

设这个问题的数学模型如下：

设从 i 到 j 的距离为d_{ij}，设一个辅助连续变量u_i，表示城市 i 在路径中的次序，$i, j = 1, 2, \cdots, n$

$$d_{ij} = \sqrt{\left(x_i - x_j\right)^2 + \left(y_i - y_j\right)^2}$$

目标函数为

$$\min \sum_{i=1}^{n} \sum_{j=1, j\neq i}^{n} c_{ij}\, d_{ij}$$

约束条件为

$$c_{ij} = \begin{cases} 1, & \text{from } i \text{ to } j \\ 0, & \text{not from } i \text{ to } j \end{cases}$$

满足出度和与入度和都为 1，即每个城市都离开或进入一次。

$$\sum_{j=1, j\neq i}^{n} c_{ij} = 1, \quad i = 1,2, \ldots, n$$

$$\sum_{i=1, i\neq j}^{n} c_{ij} = 1, \quad j = 1,2, \ldots, n$$

子回路消除约束

$$\begin{aligned} u_i - u_j + n c_{ij} &\leq n - 1, \\ i, j &= 2, 3, \ldots, n,\ i \neq j, \\ 1 &\leq u_i \leq n - 1. \end{aligned}$$

2 算法设计思路

由之前的分析可知，不能使用暴力枚举得到全部结果，因此考虑使用全局搜索算法。在《最优化原理与方法》课程中，我们学习了模拟退火算法、遗传算法和粒子群优化算法，本问题考虑使用模拟退火算法。

模拟退火算法基于固体退火原理，通过“概率接受劣解”的 Metropolis 准则跳出局部最优，在降温过程中逐步收敛到全局近似最优解，兼具全局搜索能力与可控的时间复杂度，非常适合 TSP 这类离散组合优化问题，因此选为本次实验的核心算法。

本实验的模拟退火算法流程分为五大模块：数据预处理、初始路径生成、邻域解构造、迭代寻优、结果可视化。核心逻辑是通过“贪婪初始化+随机邻域扰动+Metropolis 准则+逐步降温”的流程，在迭代中不断逼近最优路径。伪代码如下所示

算法：基于模拟退火的旅行商问题求解 输入：城市集合 Cities、城市坐标 Coordinates、起始城市 origin、超参数（初始温度 T0、降温速率α、单温度迭代次数 L、终止温度 T_end） 输出：最优路径 best_route、最短总距离 best_dist
1. 数据预处理 1.1 读取城市坐标数据，构建城市列表与坐标矩阵 1.2 计算城市间欧氏距离矩阵 Distances 1.3 获取起始城市索引 origin_index，固定路径起点
2. 初始化路径与参数 2.1 使用贪婪算法生成初始路径 current_route：从起点出发，每次选择距离最近的未访问城市 2.2 计算初始路径总距离 current_dist = total_distance(current_route) 2.3 初始化最优路径 best_route = current_route，最优距离 best_dist = current_dist 2.4 设置当前温度 T = T0，初始化迭代记录列表
3. 迭代寻优（模拟退火主循环） WHILE T > T_end: FOR i in 1 to L: 3.1 生成邻域新路径：随机交换 current_route 中两个非起点城市的位置，得到 new_route 3.2 计算新路径总距离 new_dist = total_distance(new_route) 3.3 计算距离差 delta = new_dist - current_dist 3.4 基于 Metropolis 准则判断是否接受新路径： 3.5 记录当前距离与最优距离，用于绘制收敛曲线 3.6 降温操作：T = T * α
4. 结果输出与可视化 4.1 打印最优路径的城市访问顺序与最短距离 4.2 绘制最优路径的城市分布图与路线图 4.3 绘制迭代过程的距离收敛曲线 4.4 保存结果文件与可视化图像

3 核心代码说明

本实验代码封装为 SimulatedAnnealingTSP 类，核心模块与关键实现说明如下。

3.1 类初始化模块

```
def __init__(self, cities, coordinates, origin="北京"):
        self.origin = origin  # 起始城市
        self.cities = cities  # 加载城市数据
        self.coordinates = np.asarray(coordinates, dtype=np.float64)# 加载城市坐标数据
        self.distances = (self._cal_matrix_distance())
        # 城市距离矩阵，并转换为 NumPy 数组以加速计算
        # 模拟退火算法超参数
        self.settings = Settings()
```

初始化各参数，导入城市名称、坐标并计算对称的距离矩阵；在外部 settings.py 中设置模拟退火算法的超参数，具体如下。

```
class Settings:
    def __init__(self):
        self.initial_temp = 5000.0  # 初始温度
        self.cooling_rate = 0.99    # 降温速率（每轮乘以该系数）
        self.num_iter = 50      # 每个温度下的迭代次数
        self.stop_temp = 1e-10  # 停止温度（低于此值终止）
```

3.2 数据预处理模块

```
def _cal_matrix_distance(self):
        # 利用广播计算坐标差: shape (N, N, 2)
        diff = self.coordinates[:, np.newaxis, :] - self.coordinates[np.newaxis, :, :]
        # 计算欧氏距离: shape (N, N)
        dist = np.sqrt(np.sum(diff**2, axis=-1))
        return dist
```

本模块采用 NumPy 广播机制向量化计算所有城市对的欧氏距离，避免循环嵌套，时间复杂度从 $O(n^2)$的循环优化为底层 C 实现的向量化运算，大幅提升距离计算效率。

3.3 初始路径生成模块

```
def _greed_route(self):
        unvisited = set(range(self.num_cities))
        route = [self.origin_index]
```

```
        while unvisited:
            last = route[-1]
            next_city = min(unvisited, key=lambda c: self.distances[last][c])
            route.append(next_city)
            unvisited.remove(next_city)
        return route
```

本模块采用贪婪算法生成初始路径，相比随机初始路径，能快速得到一个较优的初始解，减少后续迭代的搜索成本，加速算法收敛。

3.4 邻域解构造模块

```
def _neighour_route(self, route):
        new_route = route.copy()
        idx1, idx2 = random.sample(range(1, self.num_cities), 2)
        new_route[idx1], new_route[idx2] = new_route[idx2], new_route[idx1]
        return new_route
```

本模块通过随机交换两个非起点城市的位置生成邻域解，既保证了路径的合法性（起点固定、每个城市仅访问一次），又实现了对当前路径的有效扰动，为跳出局部最优提供基础。

3.5 模拟退火主模块

```
def simulated_annealing(self):
        # 1. 初始化当前解和最优解
        current_route = self._greed_route()  # 使用贪婪算法生成初始路径
        current_dist = self._total_distance(current_route)  # 计算初始路径距离
        best_route = current_route.copy()  # 最优路径
        best_dist = current_dist  # 最优路径距离
        temperature = self.initial_temp  # 初始温度
        # 2. 执行模拟退火算法
        while temperature > self.stop_temp:
            for _ in range(self.num_iter):
                # 生成新解
                new_route = self._neighour_route(current_route)
                new_dist = self._total_distance(new_route)
                # 计算距离差
                delta = new_dist - current_dist
                # 判断是否接受新解
                if delta < 0 or random.random() < math.exp(-delta / temperature):
                    # Metropolis 准则
                    current_route = new_route
                    current_dist = new_dist
```

```
                    # 更新最优解
                    if current_dist < best_dist:
                        best_route = current_route.copy()
                        best_dist = current_dist
            # 3. 降温
            temperature *= self.cooling_rate
        return best_route, best_dist
```

本模块实现模拟退火算法，用于求解最短访问路径问题。算法以贪心策略生成初始解，通过邻域操作生成新路径，基于 Metropolis 准则，即令

$$\Delta = \text{new_dist} - \text{current_dist}$$

如果$\Delta < 0$，直接接受新解作为最优解；

如果$\Delta \geq 0$，依据概率

$$P = \mathrm{e}^{-\Delta/T}$$

决定是否接受新解，若当前温度较大，则接受新解的概率更大，有利于前期全局搜索，避免陷入局部最优；随着温度降低，接受新解的概率减小，适应后期的局部优化，除非新解明显优于当前最优解，否则几乎不接受新解。

最终输出最优路径及最短距离，并记录迭代数据用于分析。

3.6 可视化模块

```
def print_route(self, route):
    """打印访问路径"""
```

```
def plot_route(self, route, best_dist, save_path="."):
    """可视化 TSP 最优路径：带城市名称、方向箭头、起点高亮"""
```

```
def plot_dist_curve(self, best_dist, save_path="."):
    """绘制每次迭代的距离变化曲线，即损失函数的收敛情况"""
```

实现了路径可视化与收敛曲线可视化，直观呈现最优路径的访问顺序与算法的收敛过程，便于分析算法性能。

4 实验结果展示和说明

本节实验设置固定随机种子，保证结果可复现。优化前和优化后的超参数及主要运行结果见表 4.1。其中，运行时间是多次运行后的估计值，最短距离为可复现结果。

表 4.1 模拟退火算法超参数设置和运行结果

超参数	未优化前	优化后
初始温度	5000	5000
降温速率	0.997	0.99
单温度迭代次数	500	50
运行时间	大于 100s	小于 5s
最短距离	16845.87	17450.45

4.1 未优化超参数前

运行结果如下

```
城市数量: 34
城市列表:
北京      天津      上海      重庆      拉萨      乌鲁木齐  银川      呼和浩特  南宁      哈尔滨
长春      沈阳      石家庄    太原      西宁      济南      郑州      南京      合肥      杭州
福州      南昌      长沙      武汉      广州      台北      海口      兰州      西安      成都
贵阳      昆明      香港      澳门
============================================================
运行时间: 110.71秒
最短访问路径:
北京 ->     天津 ->     沈阳 ->     长春 ->     哈尔滨 ->   呼和浩特 -> 银川 ->     乌鲁木齐 -> 拉萨 ->     西宁 ->
兰州 ->     太原 ->     济南 ->     郑州 ->     合肥 ->     南京 ->     武汉 ->     长沙 ->     南昌 ->     上海 ->
杭州 ->     福州 ->     台北 ->     香港 ->     澳门 ->     广州 ->     海口 ->     南宁 ->     贵阳 ->     昆明 ->
重庆 ->     成都 ->     西安 ->     石家庄 ->   北京

最短距离: 16845.87
最优路径图已保存至: .\tsp_best_route_distance_16845.87.png
距离曲线图已保存至: .\tsp_distance_curve_distance_16845.87.png
```

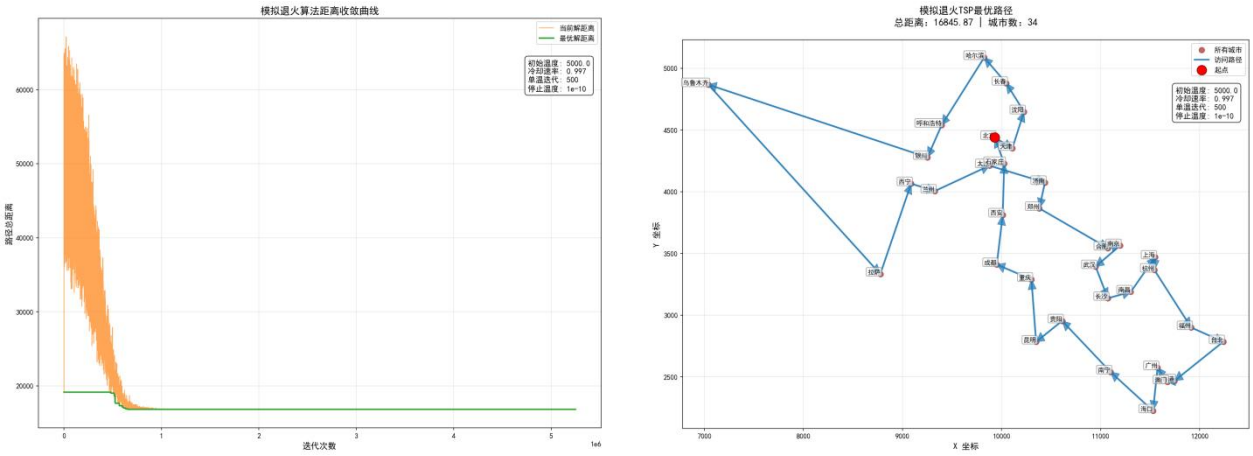


图 4.1 未优化前运行结果

从图中可清晰看出，贪心算法得到的初始路线代价（路径总距离）已低于 20000；但全局搜索方法在迭代初期探索了大量代价更高的解，导致高温阶段的代价函数几乎无下降趋势。随着迭代持续推进，代价函数逐步下降，并在约 10^6 次迭代时趋于收敛，后续优化空间极小，可视为无效迭代阶段。基于此，需调整算法超参数，通过减少迭代次数、加快降温速率，提升算法优化效率。

4.2 优化超参数后

运行结果如下

```
城市数量: 34
城市列表:
北京      天津      上海      重庆      拉萨      乌鲁木齐  银川      呼和浩特  南宁      哈尔滨
长春      沈阳      石家庄    太原      西宁      济南      郑州      南京      合肥      杭州
福州      南昌      长沙      武汉      广州      台北      海口      兰州      西安      成都
贵阳      昆明      香港      澳门
============================================================
运行时间: 3.86秒
最短访问路径:
北京 ->    呼和浩特 -> 银川 ->    乌鲁木齐 -> 拉萨 ->    西宁 ->    兰州 ->    西安 ->    武汉 ->    合肥 ->
南京 ->    长沙 ->    南昌 ->    上海 ->    杭州 ->    福州 ->    台北 ->    香港 ->    澳门 ->    广州 ->
海口 ->    南宁 ->    贵阳 ->    昆明 ->    重庆 ->    成都 ->    郑州 ->    济南 ->    石家庄 ->  太原 ->
天津 ->    沈阳 ->    长春 ->    哈尔滨 ->  北京

最短距离: 17450.45
最优路径图已保存至: .\tsp_best_route_distance_17450.45.png
距离曲线图已保存至: .\tsp_distance_curve_distance_17450.45.png
```

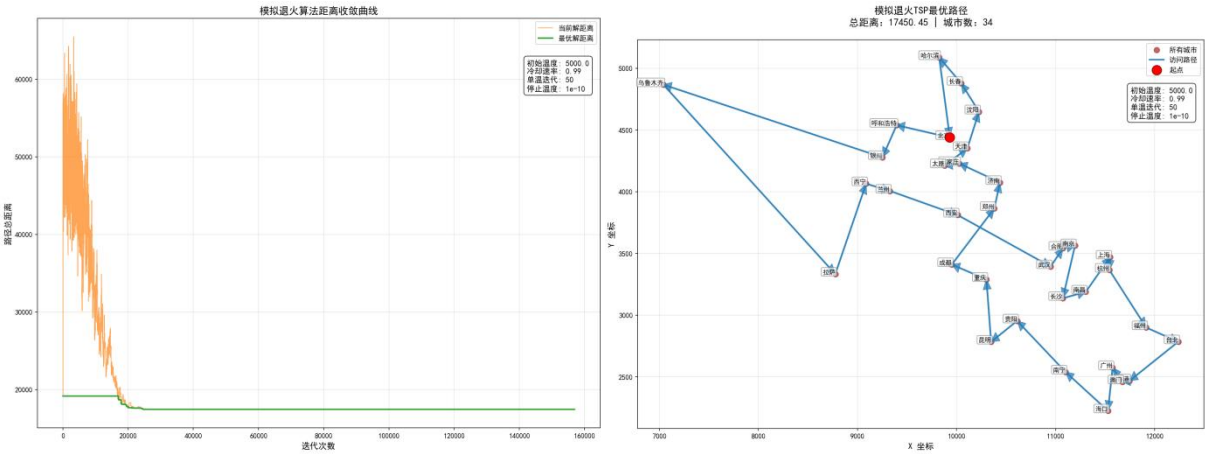


图 4.2 优化后运行结果

由图可知，优化后算法在迭代 20000 次左右，代价函数开始呈现明显下降趋势，并逐步趋于收敛；同时，算法运行时间从约 110 秒大幅缩减至约 4 秒，效率得到显著提升。但需注意的是，优化后的路径总距离略高于优化前，即该优化是以牺牲部分收敛精度为代价，换取了大量时间成本的节省，实现了时间与精度的权衡。

4.3 对比随机路径和贪心路径

在 3.3 节中提到，代码采用贪心算法生成初始路径，能够有效加快算法收敛速度，但初始路径本身已为较优解，会使模拟退火从较优值开始搜索，一定程度上削弱了算法自身的智能搜索特性。为更真实地研究模拟退火算法的原始收敛特性，本实验对初始路径生成方式进行修改，采用随机法生成初始路径，具体实现如下。

```
def _random_route(self):
        other_cities = [i for i in range(self.num_cities) if i != self.origin_index]
```

```
        random.shuffle(other_cities)
        route = [self.origin_index] + other_cities
        return route
```

分别在超参数优化前、优化后进行测试。由于随机路径生成的初始总距离较大，算法在前期全局搜索阶段，最优路径距离随迭代过程呈现明显的阶梯式下降；进入后期后，距离逐渐收敛，算法转为局部优化，整体过程符合全局优化算法的典型特征。

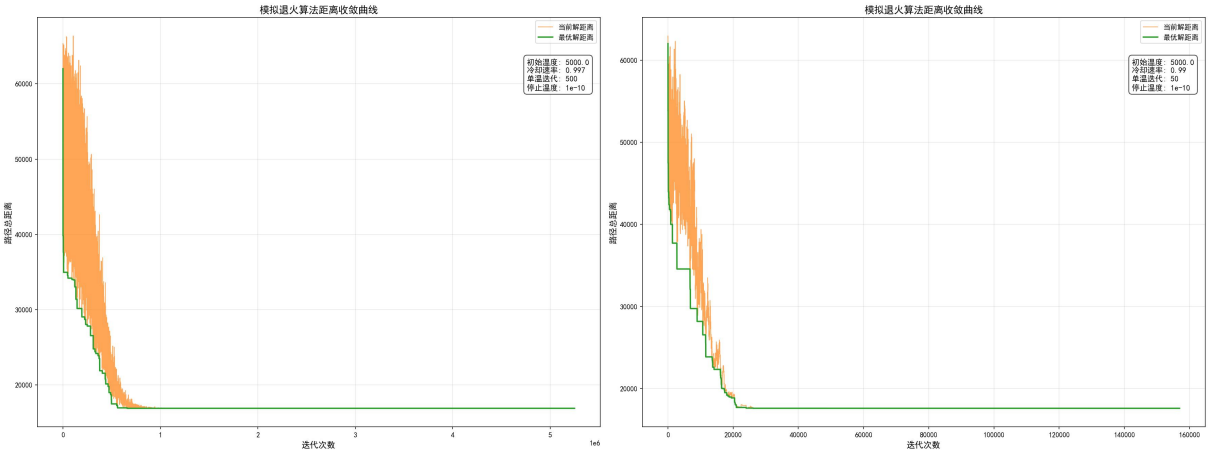


图 4.3 随机路径生成运行结果

4.4 寻找最优解

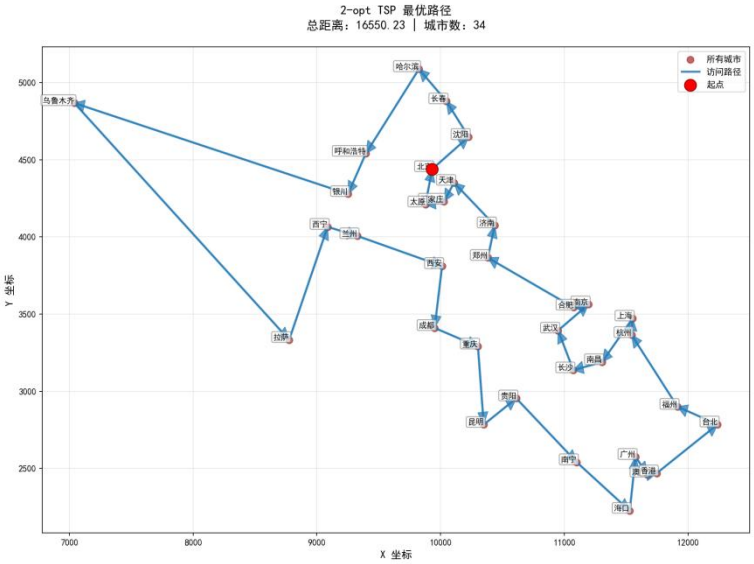
为精准求解本问题，引入 2-opt 算法作为对比方案。该算法属于基于路径片段反转的局部搜索策略，首先采用最近邻贪心算法构建初始可行路径，再通过不断反转局部路径段进行迭代优化，持续缩减回路总距离。将其与模拟退火算法横向对比，能够直观分析模拟退火在全局寻优性能与算法运行效率之间的取舍与平衡关系。

运行结果如图 4.4 所示。

经 2-opt 算法求解得到本次旅行商问题的最优路径，最短总距离为 16550.23。该算法逻辑简洁、运算高效，时间成本极低，但仅依靠局部搜索策略迭代优化，

```
城市数量: 34
城市列表:
北京    天津    上海    重庆    拉萨    乌鲁木齐  银川    呼和浩特  南宁    哈尔滨
长春    沈阳    石家庄  太原    西宁    济南    郑州    南京    合肥    杭州
福州    南昌    长沙    武汉    广州    台北    海口    兰州    西安    成都
贵阳    昆明    香港    澳门
============================================================
算法运行时间: 2.52 秒
2-opt 算法求解 TSP 最优路径
最短距离: 16550.23
最优路径:
北京 ->    沈阳 ->    长春 ->    哈尔滨 ->  呼和浩特 -> 银川 ->    乌鲁木齐 -> 拉萨 ->    西宁 ->    兰州 ->
西安 ->    成都 ->    重庆 ->    昆明 ->    贵阳 ->    南宁 ->    海口 ->    广州 ->    澳门 ->    香港 ->
台北 ->    福州 ->    杭州 ->    上海 ->    南昌 ->    长沙 ->    武汉 ->    南京 ->    合肥 ->    郑州 ->
济南 ->    天津 ->    石家庄 ->  太原 ->    北京

最优路径图已保存至: .\tsp_best_route_distance_16550.23_2-opt.png
```



容易收敛至局部最优解，全局寻优能力有限。

图 4.4 2-opt 算法运行结果

5 性能分析

5.1 时间复杂度分析

在算法预处理阶段，城市距离矩阵的构建过程时间复杂度为$O(n^2)$；初始路径由贪心算法生成，其时间复杂度同样为$O(n^2)$。算法主循环中，设单一温度下的迭代次数为 L，整体降温迭代次数由降温系数决定，数量级为$\log_\alpha(T_{end}/T_0)$。综上，模拟退火算法整体时间复杂度可表示为$O\left(n^2 \times L \cdot \log(T_{end}/T_0)\right)$。本次实验城市数量 n=34，设置的初始温度和停止温度为常数，因此，本实验实际时间复杂度为$O(n^2)$，运行耗时约 10^0 秒级别，相较于暴力枚举法，计算开销大幅降低，求解效率优势显著。

实验结果表明，算法在迭代后期会进入较长的收敛平台期，路径总距离几乎不再下降，造成不必要的计算资源浪费。为解决这一问题，可引入早停机制：当连续多轮迭代未找到更优路径时，提前终止算法。但该机制存在一定风险，若参数设置不当，可能导致算法在前期全局搜索阶段尚未充分探索、未找到更优解时就提前终止。因此，需合理设置早停阈值，在求解精度与迭代效率之间取得平衡。

5.2 算法性能影响因素

初始温度、降温速率与单温度迭代次数是影响模拟退火性能的核心超参数。初始温度越高，算法接受劣解的概率越大，全局探索能力越强，但整体迭代耗时

随之增加；降温速率过慢会拉长迭代周期、提升运算耗时，降温速率过快则会过早收敛，易陷入局部最优解；各温度下的迭代次数越多，局部邻域搜索越充分，解的质量更优，但会进一步提升整体计算成本。实际应用中需对各项参数进行合理调配，平衡算法搜索能力与运行效率。

5.3 横向对比分析

2-opt 算法时间复杂度低，具有计算量小、速度快、易于实现的优点，但该算法仅基于局部搜索，易收敛至局部最优解。由于本次实验数据集规模较小、分布规律较好，局部极小值点较少，因此 2-opt 算法在本问题中表现出较高的求解精度。在实际工程应用中，应根据场景需求选择合适算法：对实时性要求高、精度要求较低的场景，可优先使用 2-opt 算法；对全局最优性要求较高的场景，更适合采用模拟退火算法。此外，还可将 2-op 嵌入模拟退火框架用于邻域解优化，结合模拟退火的全局搜索能力与 2-opt 的局部求精能力，实现求解精度与运行效率的最优平衡。

6 总结与讨论

本次实验以 34 个城市旅行商问题为研究对象，针对暴力枚举复杂度过高的问题，采用模拟退火算法开展路径优化求解。实验证明，该算法依托 Metropolis 准则，有效跳出局部最优，通过调节初始温度、降温速率等关键参数，平衡算法精度与运行速度，参数优化后运算效率显著提升，更贴合实际工程应用。与 2-opt 局部搜索算法相比，模拟退火全局搜索能力更强，更适用于 TSP 这类 NP 难优化问题。本实验同时表明智能优化算法在组合优化问题中的实用价值，未来可将模拟退火与 2-opt 局部搜索结合，构建混合优化算法，进一步平衡求解精度与运行效率。整体而言，模拟退火算法稳定性与适应性良好，可为物流规划、资源调度等实际工程场景提供可靠参考。